

emmanuel_ansah_SML1

Emmanuel Ansah

12/7/2024

Packages

```
library(ggplot2)
library(caret)
```

```
## Loading required package: lattice
```

```
library(kernlab)
```

```
##
```

```
## Attaching package: 'kernlab'
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##      alpha
```

```
library(MASS)
library(e1071)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

Exercise 1:

Part 1: Dataset Exploration

1. Download the file aims-sml-2024-2025-data.csv and load it into RMarkdown.

```
mldata=read.csv("aims-sml-2024-2025-data.csv")
attach(mldata)
```

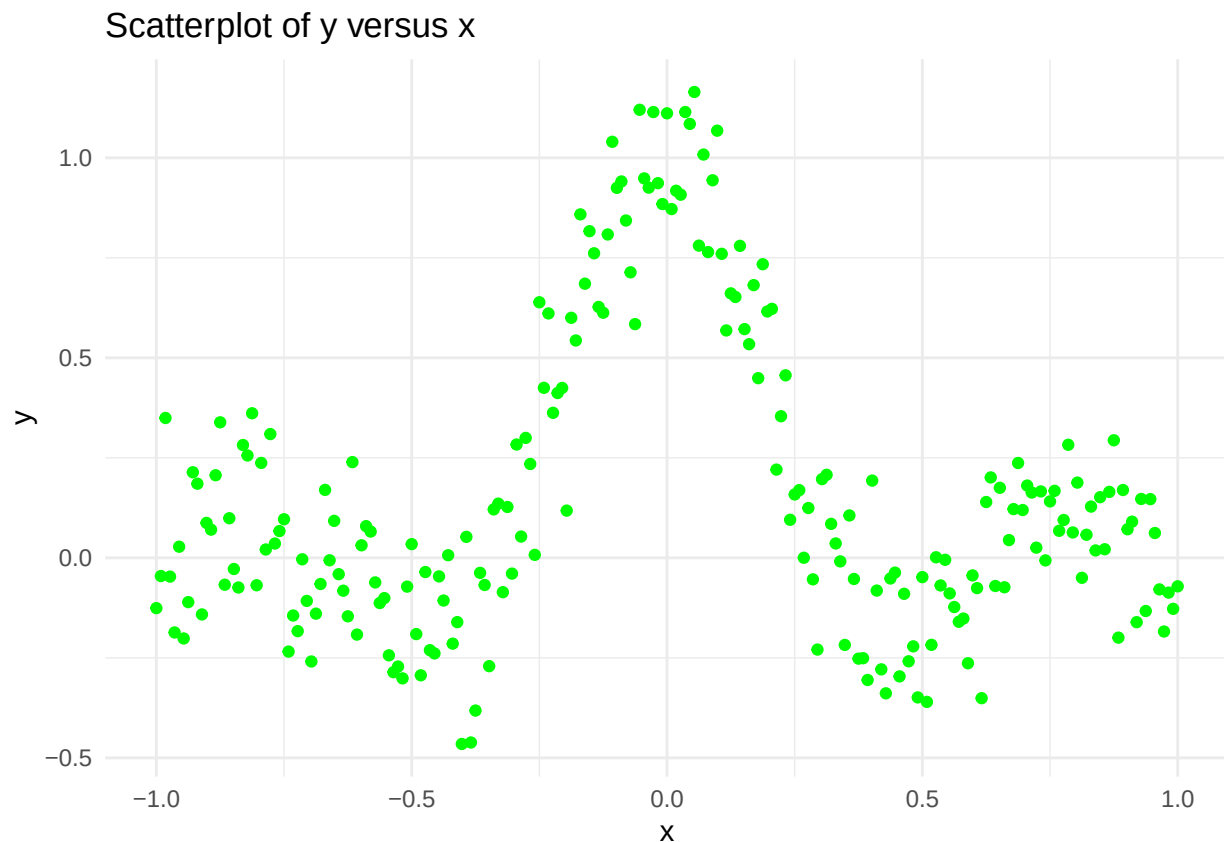
2. Determine the size of the dataset n.

```
n = nrow(mldata)
p= ncol(mldata)
cat("Data has",n,"rows and ",p,"columns" )
```

```
## Data has 225 rows and 2 columns
```

3. Create a scatterplot of y versus x.

```
ggplot(mldata, aes(x = x, y = y)) +
  geom_point(color = "green") +
  labs(
    title = "Scatterplot of y versus x",
    x = "x",
    y = "y"
  ) +
  theme_minimal()
```



4. Determine whether this is a classification or regression task, and justify your answer.

```
str(mldata)
```

```
## 'data.frame': 225 obs. of 2 variables:
## $ x: num -1 -0.991 -0.982 -0.973 -0.964 ...
## $ y: num -0.1257 -0.0455 0.3497 -0.0469 -0.1865 ...
```

```
summary(mldata)
```

```
##           x           y
## Min.      :-1.0    Min.      :-0.46525
## 1st Qu.: -0.5    1st Qu.: -0.08579
## Median :  0.0    Median :  0.06703
## Mean     :  0.0    Mean      :  0.16620
## 3rd Qu.:  0.5    3rd Qu.:  0.30949
## Max.     :  1.0    Max.       :  1.16462
```

This is a regression task. Justification: In regression tasks, the target variable is continuous, whereas in classification tasks, the target variable would be categorical. Since the response variable y is continuous, this is a regression task.

Part 2: Theoretical Framework

1. Suggest a function space $\mathcal{H}$ for this task and write it in mathematical notation.

The function space $\mathcal{H}$ for this regression task is the space of polynomial functions of degree p .

$$\mathcal{H} = \left\{ f(x) = \sum_{i=0}^p \beta_i x^i \mid \beta_i \in \mathbb{R}, p \in \mathbb{N} \right\}$$

Where:

- $f(x)$ is a polynomial function that maps the input x to the predicted output y
- β_i are the coefficients of the polynomial
- p is the degree of the polynomial, which can be adjusted based on model selection

2. Specify the loss function for this task and justify its use.

The loss function for this regression task is the **Mean Squared Error (MSE)**, defined as:

$$L(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the actual value and $\hat{y}_i$ is the predicted value. \ **Justification:**

- **Minimizes errors:** Penalizes large errors more than small ones, encouraging a better fit.
- **Smooth and differentiable:** Suitable for optimization algorithms like gradient descent.
- **Statistical foundation:** MSE is derived from maximum likelihood estimation under the assumption of normal errors.

3. Derive and write down the theoretical risk $R(f)$ for a candidate function f in $\mathcal{H}$.

The **theoretical risk** $R(f)$ for a candidate function $f \in \mathcal{H}$ is defined as the expected value of the loss function over the distribution of the data. For a regression problem with the Mean Squared Error (MSE) loss, it is given by:

$$R(f) = \mathbb{E}[(Y - f(X))^2]$$

where:

- X is the input (independent variable),
- Y is the true output (dependent variable),
- $f(X)$ is the predicted output from the model,
- $\mathbb{E}[\cdot]$ denotes the expectation with respect to the joint distribution of (X, Y) .

$$R(f) = \int (y - f(x))^2 p(x, y) dx dy$$

where $p(x, y)$ is the joint probability density function of X and Y .

4. Write down the expression for the Bayes learning machine $f^*(x)$ in this case.

The Bayes learning machine $f^*(x)$ is the optimal predictor that minimizes the expected loss (or risk) in a regression task. In the context of regression with a squared loss function, $f^*(x)$ is the conditional expectation of the output Y given the input $X = x$,

$$f^*(x) = \mathbb{E}[Y | X = x]$$

where $\mathbb{E}[Y | X = x]$ is the conditional expectation of Y given $X = x$, which can be estimated using the joint distribution of X and Y .

5. Write down the empirical risk $\hat{R}(f)$ for a candidate function, including all components of the model space. Hint: Decide on a complexity prior to training.

The **empirical risk** $\hat{R}(f)$ for a candidate function f is given by:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \mathcal{C}(f)$$

where:

- n is the number of data points,

- (x_i, y_i) are the input-output pairs from the dataset,
- $f(x_i)$ is the predicted output from the model for input x_i ,
- y_i is the true output for the corresponding input x_i ,
- λ is a regularization parameter that controls the trade-off between the model's complexity and its fit to the data,
- $\mathcal{C}(f)$ is a complexity measure of the function f , which could depend on the type of model.

The first term represents the Mean Squared Error (MSE), and the second term, the regularization term, penalizes the complexity of the model to avoid overfitting.

Part 3: Estimation and Model Complexity

1. Derive the expression for the OLS estimator $\hat{f}(x)$ for this problem.

General Form of the Model We assume the relationship between y and x is given by a polynomial regression model of degree p :

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_p x_i^p + \epsilon_i, \quad i = 1, 2, \dots, n$$

where:

- y_i is the observed response variable,
- x_i is the predictor variable,
- ϵ_i is the error term with $\mathbb{E}[\epsilon_i] = 0$ and $\text{Var}(\epsilon_i) = \sigma^2$,
- $\beta = [\beta_0, \beta_1, \dots, \beta_p]^\top$ are the coefficients to be estimated.

Minimizing the Residual Sum of Squares (RSS) The residual for each observation is:

$$r_i = y_i - (\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_p x_i^p)$$

The Residual Sum of Squares (RSS) is:

$$RSS = \sum_{i=1}^n r_i^2 = \sum_{i=1}^n \left(y_i - \sum_{j=0}^p \beta_j x_i^j \right)^2$$

Matrix Representation

$$\mathbf{y} = \mathbf{X}\beta + \epsilon$$

where:

- $\mathbf{y} = [y_1, y_2, \dots, y_n]^\top$ (response vector),
- $\beta = [\beta_0, \beta_1, \dots, \beta_p]^\top$ (coefficients vector),
- $\epsilon = [\epsilon_1, \epsilon_2, \dots, \epsilon_n]^\top$ (error vector),
- $\mathbf{X}$ is the design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{bmatrix}$$

RSS in Matrix Form

$$RSS = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

Expanding,

$$RSS = \mathbf{y}^\top \mathbf{y} - 2\boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta}$$

To minimize the RSS, we take the derivative with respect to $\boldsymbol{\beta}$ and set it to zero:

$$\frac{\partial RSS}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} = 0$$

Simplifying:

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$$

The solution for $\boldsymbol{\beta}$ is given by the normal equation:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Using $\hat{\boldsymbol{\beta}}$, the predicted response at a given x is:

$$\hat{f}(x) = \mathbf{x}^\top \hat{\boldsymbol{\beta}}$$

where $\mathbf{x} = [1, x, x^2, \dots, x^p]^\top$.

2. Comment extensively on the properties of $\hat{f}(\mathbf{x})$

Unbiasedness: The OLS estimator is unbiased under the assumption that the model is correctly specified. This means that, on average, the OLS estimator will predict the true values of y . Specifically, if the relationship between x and y is linear, the expected value of the estimator $\hat{f}(x)$ is equal to the true function $f(x)$:

$$\mathbb{E}[\hat{f}(x)] = f(x)$$

This property holds as long as the noise term in the data is zero-mean and the predictors are not correlated with the error term.

Efficiency: The OLS estimator is efficient in the sense that it minimizes the variance among all linear estimators of the form $\hat{f}(x) = \mathbf{X}\boldsymbol{\beta}$, where $\mathbf{X}$ is the design matrix and $\boldsymbol{\beta}$ are the coefficients. This efficiency is achieved when the errors are homoscedastic (constant variance) and uncorrelated.

Consistency: The OLS estimator is consistent, meaning that as the sample size n increases, the estimator $\hat{f}(x)$ converges to the true function $f(x)$ if the model is correctly specified and the data satisfies the conditions of the Law of Large Numbers. Specifically, we have:

$$\hat{f}(x) \xrightarrow{P} f(x) \text{ as } n \rightarrow \infty$$

This means that with a large enough sample, the OLS estimator will get arbitrarily close to the true function.

Best Linear Unbiased Estimator (BLUE): Under the Gauss-Markov assumptions, which include linearity, homoscedasticity, and uncorrelated errors, the OLS estimator is the Best Linear Unbiased Estimator (BLUE). This means that it is the most efficient (having the smallest variance) among all unbiased linear estimators.

Sensitivity to Outliers: The OLS estimator can be highly sensitive to outliers in the data. Since the OLS method minimizes the sum of squared errors, large deviations from the true function (outliers) can disproportionately influence the estimator. This is a potential limitation when working with real-world data that may contain outliers.

Overfitting and Underfitting:

- **Overfitting:** If the degree of the polynomial is too high (i.e., too many parameters are included in the model), the OLS estimator may overfit the data, capturing noise rather than the underlying relationship between x and y .
- **Underfitting:** On the other hand, if the model is too simple (e.g., using a linear function for a clearly non-linear relationship), the OLS estimator may underfit the data, failing to capture the true relationship between x and y .

Interpretability: The OLS estimator provides an explicit relationship between the input variables and the predicted output. In a polynomial regression model, the coefficients of the polynomial (i.e., the β_i 's) represent the contribution of each power of x to the model. This makes the OLS estimator interpretable in terms of the effects of each feature.

Computational Complexity: The computation of the OLS estimator involves solving a linear system or inverting the design matrix. For small datasets, this is computationally feasible. However, for large datasets with many predictors, it can be computationally expensive, especially if the design matrix is ill-conditioned.

3. Use V -fold cross-validation (e.g., $V = 5, 10$) to determine the optimal complexity (degree) for the polynomial regression model.

Explain what “optimal complexity” means.

“Optimal complexity” refers to the best model that strikes a balance between fitting the data well and avoiding overfitting. In the context of polynomial regression, this means selecting the degree p of the polynomial such that the model:

- Fits the data well (low bias),
- Generalizes well to unseen data (low variance),
- Avoids overfitting by not capturing noise or irrelevant details in the data.

```
x <- mldata$x
y <- mldata$y

# Function for V-Fold Cross-Validation
v_fold_cv <- function(x, y, degree, V) {
  set.seed(08122024) # For reproducibility
```

```

# Split data indices into V folds
fold_indices <- caret::createFolds(y, k = V, list = TRUE)
validation_errors <- numeric(V)

for (v in seq_along(fold_indices)) {
  # Define the validation set
  test_indices <- fold_indices[[v]]
  x_val <- x[test_indices]
  y_val <- y[test_indices]

  # Define the training set
  x_train <- x[-test_indices]
  y_train <- y[-test_indices]

  # Build the polynomial regression model
  poly_model <- lm(y_train ~ poly(x_train, degree, raw = TRUE))

  # Prepare a data frame for prediction
  val_data <- data.frame(x_train = x_val)
  colnames(val_data) <- all.vars(poly_model$terms)[2]

  # Compute predictions for the validation set
  y_pred <- predict(poly_model, newdata = val_data)

  # Compute the validation error (Mean Squared Error)
  validation_errors[v] <- mean((y_val - y_pred)^2)
}

# Compute the average validation error
return(mean(validation_errors))
}

# Function to compute empirical risk
compute_empirical_risk <- function(x, y, degree) {
  # Fit polynomial regression model
  poly_model <- lm(y ~ poly(x, degree, raw = TRUE))

  # Make predictions on the entire dataset
  y_pred <- predict(poly_model)

  # Calculate Mean Squared Error
  mse <- mean((y - y_pred)^2)
  return(mse)
}

# Degrees and number of folds
degrees <- 1:18
folds_list <- c(5, 10, 20)

# Initialize results
results <- list()
empirical_risks <- sapply(degrees, function(degree) compute_empirical_risk(x, y, degree))

```



```

for (folds in folds_list) {
  # Perform cross-validation for each degree
  mse_vfold <- sapply(degrees, function(degree) v_fold_cv(x, y, degree, V = folds))

  # Find the optimal degree
  optimal_degree <- degrees[which.min(mse_vfold)]

  # Save results
  results[[as.character(folds)]] <- data.frame(
    Degree = degrees,
    CrossValidationError = mse_vfold,
    EmpiricalRisk = empirical_risks,
    OptimalDegree = ifelse(degrees == optimal_degree, "Optimal", "Non-Optimal")
  )

  # Print optimal degree
  cat("Optimal Degree for", folds, "Folds:", optimal_degree, "\n")
}

```

```

## Optimal Degree for 5 Folds: 10
## Optimal Degree for 10 Folds: 10
## Optimal Degree for 20 Folds: 10

```

```

# Combine results for display
all_results <- do.call(rbind, lapply(names(results), function(folds) {
  data <- results[[folds]]
  data$Folds <- as.numeric(folds)
  return(data)
}))

# Print combined results
print(all_results)

```

##	Degree	CrossValidationError	EmpiricalRisk	OptimalDegree	Folds
## 1	1	0.14436741	0.14415694	Non-Optimal	5
## 2	2	0.11648567	0.11563218	Non-Optimal	5
## 3	3	0.11744154	0.11561842	Non-Optimal	5
## 4	4	0.08678163	0.08065145	Non-Optimal	5
## 5	5	0.08689430	0.08031958	Non-Optimal	5
## 6	6	0.03819403	0.03595265	Non-Optimal	5
## 7	7	0.03864165	0.03590255	Non-Optimal	5
## 8	8	0.02561832	0.02331425	Non-Optimal	5
## 9	9	0.02585155	0.02329601	Non-Optimal	5
## 10	10	0.02378370	0.02136015	Optimal	5
## 11	11	0.02381540	0.02135268	Non-Optimal	5
## 12	12	0.02383190	0.02106145	Non-Optimal	5
## 13	13	0.02448989	0.02086746	Non-Optimal	5
## 14	14	0.02500249	0.02083000	Non-Optimal	5
## 15	15	0.02544705	0.02081784	Non-Optimal	5
## 16	16	0.02552134	0.02074399	Non-Optimal	5
## 17	17	0.02383864	0.02000533	Non-Optimal	5
## 18	18	0.02386659	0.01997922	Non-Optimal	5
## 19	1	0.14521544	0.14415694	Non-Optimal	10

## 20	2	0.11700775	0.11563218	Non-Optimal	10
## 21	3	0.11758851	0.11561842	Non-Optimal	10
## 22	4	0.08344049	0.08065145	Non-Optimal	10
## 23	5	0.08456186	0.08031958	Non-Optimal	10
## 24	6	0.03785648	0.03595265	Non-Optimal	10
## 25	7	0.03809988	0.03590255	Non-Optimal	10
## 26	8	0.02536842	0.02331425	Non-Optimal	10
## 27	9	0.02553885	0.02329601	Non-Optimal	10
## 28	10	0.02345738	0.02136015	Optimal	10
## 29	11	0.02372934	0.02135268	Non-Optimal	10
## 30	12	0.02366109	0.02106145	Non-Optimal	10
## 31	13	0.02417842	0.02086746	Non-Optimal	10
## 32	14	0.02458296	0.02083000	Non-Optimal	10
## 33	15	0.02514516	0.02081784	Non-Optimal	10
## 34	16	0.02512542	0.02074399	Non-Optimal	10
## 35	17	0.02368672	0.02000533	Non-Optimal	10
## 36	18	0.02370780	0.01997922	Non-Optimal	10
## 37	1	0.14534571	0.14415694	Non-Optimal	20
## 38	2	0.11738008	0.11563218	Non-Optimal	20
## 39	3	0.11786109	0.11561842	Non-Optimal	20
## 40	4	0.08391124	0.08065145	Non-Optimal	20
## 41	5	0.08396021	0.08031958	Non-Optimal	20
## 42	6	0.03808010	0.03595265	Non-Optimal	20
## 43	7	0.03841620	0.03590255	Non-Optimal	20
## 44	8	0.02507461	0.02331425	Non-Optimal	20
## 45	9	0.02529963	0.02329601	Non-Optimal	20
## 46	10	0.02313171	0.02136015	Optimal	20
## 47	11	0.02336392	0.02135268	Non-Optimal	20
## 48	12	0.02330512	0.02106145	Non-Optimal	20
## 49	13	0.02364361	0.02086746	Non-Optimal	20
## 50	14	0.02409317	0.02083000	Non-Optimal	20
## 51	15	0.02461083	0.02081784	Non-Optimal	20
## 52	16	0.02478332	0.02074399	Non-Optimal	20
## 53	17	0.02316560	0.02000533	Non-Optimal	20
## 54	18	0.02348547	0.01997922	Non-Optimal	20

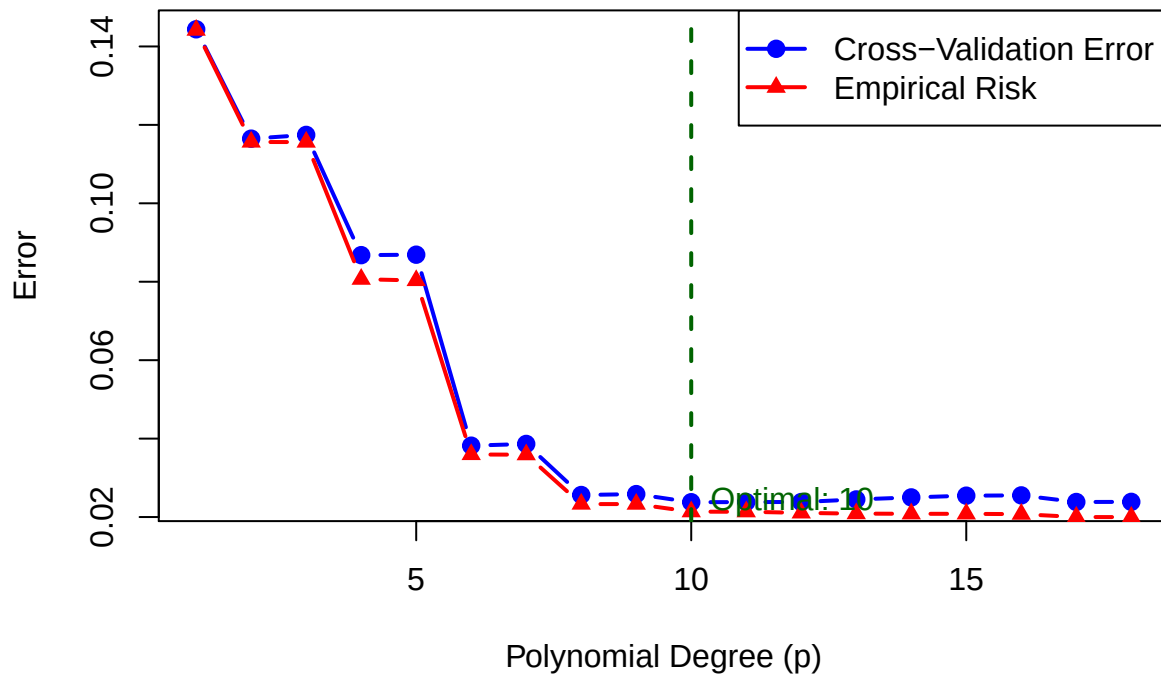
4. Plot the cross-validation error and empirical risk as functions of p . Comment on the plot.

```
# Plot Cross-Validation Error and Empirical Risk for Each Fold
for (folds in folds_list) {
  data <- results[[as.character(folds)]]
  optimal_degree <- data$Degree[which.min(data$CrossValidationError)]

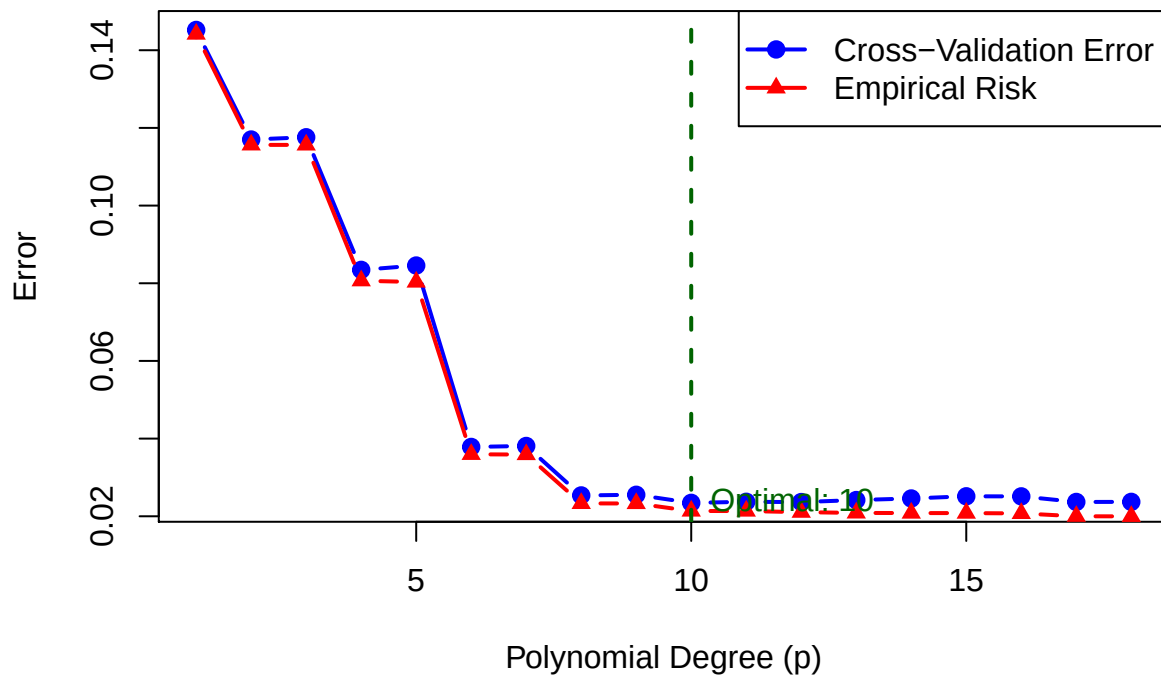
  plot(data$Degree, data$CrossValidationError, type = "b", col = "blue", pch = 19, lwd = 2,
       xlab = "Polynomial Degree (p)", ylab = "Error",
       main = paste(folds, "-Fold Cross-Validation vs Empirical Risk"))
  lines(data$Degree, data$EmpiricalRisk, type = "b", col = "red", pch = 17, lwd = 2)
  legend("topright", legend = c("Cross-Validation Error", "Empirical Risk"),
       col = c("blue", "red"), pch = c(19, 17), lwd = 2)
  abline(v = optimal_degree, col = "darkgreen", lty = 2, lwd = 2)
  text(optimal_degree, min(data$CrossValidationError), labels = paste("Optimal:", optimal_degree),
```

```
col = "darkgreen", pos = 4)
}
```

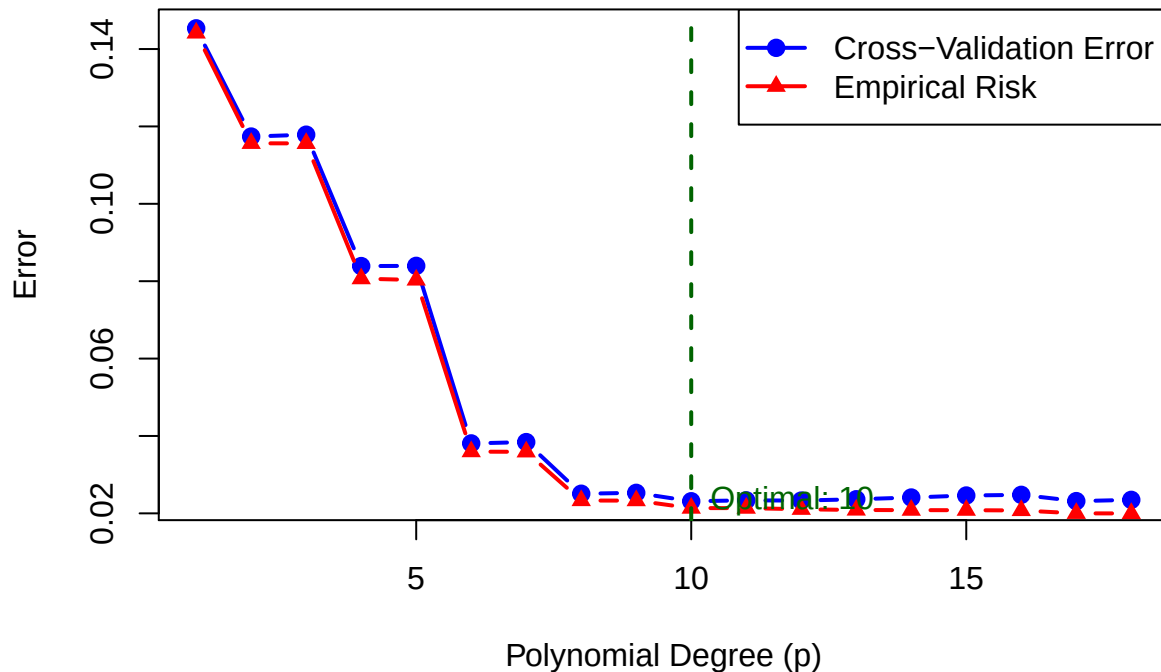
5 -Fold Cross-Validation vs Empirical Risk



10 -Fold Cross-Validation vs Empirical Risk



20 –Fold Cross–Validation vs Empirical Risk



Comment on the plot. The plot shows that both Cross-Validation Error and Empirical Risk decrease rapidly for lower degrees ($p < 7$) and stabilize around $p = 10$, which is identified as the optimal degree. The alignment between the two curves indicates consistency between training and validation, suggesting the model generalizes well. Higher degrees ($p > 10$) offer minimal improvement and may lead to unnecessary complexity. ###

Part 4: Model Comparison and Evaluation

1. Fit and plot the following models on the same plot with the data:

```
simplest_model <- lm(y ~ 1)

optimal_degree <- 10 # Based on previous cross-validation
optimal_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE))

complex_degree <- 18
complex_model <- lm(y ~ poly(x, complex_degree, raw = TRUE))

# Predictions for plotting
x_vals <- seq(min(x), max(x), length.out = 100)
simplest_pred <- predict(simplest_model, newdata = data.frame(x = x_vals))
optimal_pred <- predict(optimal_model, newdata = data.frame(x = x_vals))
complex_pred <- predict(complex_model, newdata = data.frame(x = x_vals))

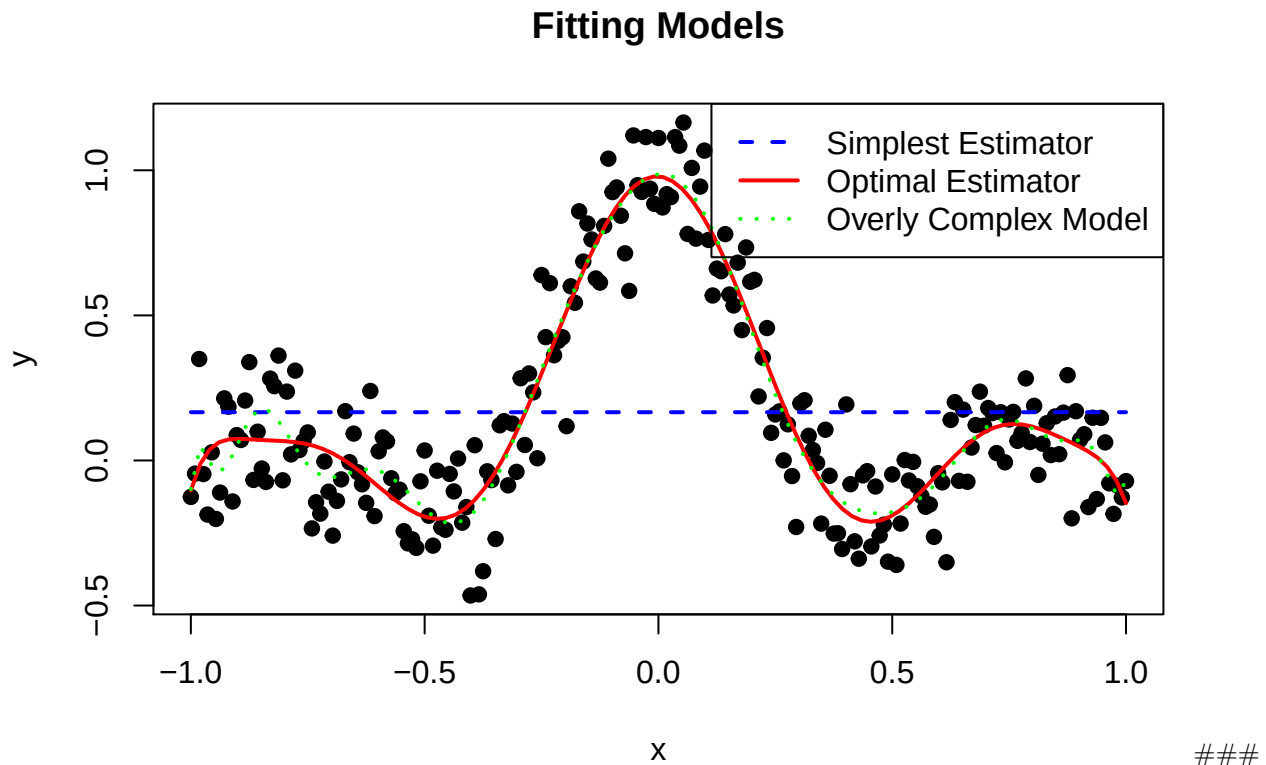
# Plotting the data and models
plot(x, y, main = "Fitting Models", xlab = "x", ylab = "y", pch = 19, col = "black")
lines(x_vals, simplest_pred, col = "blue", lwd = 2, lty = 2)
```

```

lines(x_vals, optimal_pred, col = "red", lwd = 2)
lines(x_vals, complex_pred, col = "green", lwd = 2, lty = 3)

# Add a legend
legend("topright", legend = c("Simplest Estimator", "Optimal Estimator", "Overly Complex Model"),
      col = c("blue", "red", "green"), lwd = 2, lty = c(2, 1, 3))

```



comment:

- **Simplest Estimator (Blue Dashed Line):** Underfits the data by ignoring its structure, leading to high bias and poor representation of the true pattern.
- **Optimal Estimator (Red Solid Line):** Captures the true trend accurately, balancing bias and variance, and generalizes well to unseen data.
- **Overly Complex Model (Green Dotted Line):** Overfits the data by modeling noise, resulting in low bias but high variance, which harms predictive performance on new data.

2. Perform stochastic hold-out validation with $S = 100$ splits (70% training, 30% testing).

Compute and plot boxplots of the test errors for:

```

stochastic_hold_out <- function(x, y, S = 100, M = 1, tau = 0.7, model_type = "simple") {
  set.seed(123) # For reproducibility
  errors <- numeric(S) # To store errors for each split

```

```

for (s in 1:S) {
  # Step 1: Randomly split the data into training (D_tr) and testing (D_te) sets
  n <- length(x)
  train_indices <- sample(1:n, size = floor(tau * n))
  x_train <- x[train_indices]
  y_train <- y[train_indices]
  x_test <- x[-train_indices]
  y_test <- y[-train_indices]

  # Step 2: Fit the model based on the model_type argument
  if (model_type == "simple") {
    # Simple model: Linear regression (degree = 1)
    model <- lm(y_train ~ x_train)
  } else if (model_type == "optimal") {
    # Optimal model: Polynomial regression (degree = 10)
    model <- lm(y_train ~ poly(x_train, 10, raw = TRUE))
  } else if (model_type == "complex") {
    # Complex model: Polynomial regression (degree = 18)
    model <- lm(y_train ~ poly(x_train, 18, raw = TRUE))
  }

  # Step 3: Compute predictions on the test set
  y_pred <- predict(model, newdata = data.frame(x_train = x_test))

  # Step 4: Compute the test error (Mean Squared Error)
  errors[s] <- mean((y_test - y_pred)^2)
}

# Return the errors for all splits
return(errors)
}

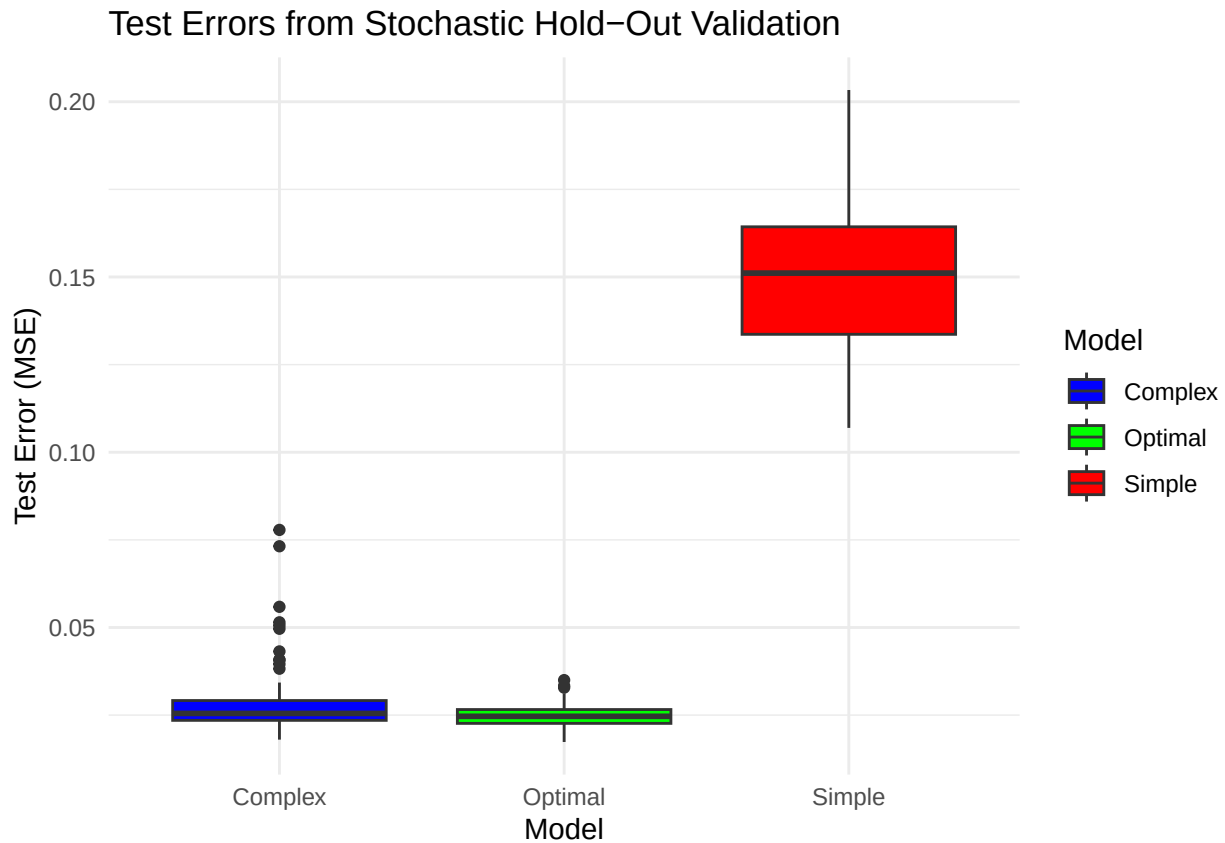
S <- 100 # Number of splits
tau <- 0.7 # Training set

# Run for the simplest, optimal, and overly complex models
errors_simple <- stochastic_hold_out(x, y, S, model_type = "simple", tau = tau)
errors_optimal <- stochastic_hold_out(x, y, S, model_type = "optimal", tau = tau)
errors_complex <- stochastic_hold_out(x, y, S, model_type = "complex", tau = tau)

# Combine the results into a data frame for plotting
results <- data.frame(
  Model = rep(c("Simple", "Optimal", "Complex"), each = S),
  Error = c(errors_simple, errors_optimal, errors_complex)
)

# Boxplot of the test errors from the S splits
ggplot(results, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot() +
  labs(title = "Test Errors from Stochastic Hold-Out Validation",
       x = "Model", y = "Test Error (MSE)") +
  theme_minimal() +
  scale_fill_manual(values = c("blue", "green", "red"))

```



In summary, the Optimal model strikes a good balance between complexity and performance, achieving the lowest test errors with high consistency. The Complex model is a close second, while the Simple model is less effective for this dataset.

Part 5: Further Analysis

1. Perform an analysis of variance (ANOVA) on the test errors.

```
anova_result <- aov(Error ~ Model, data = results)
summary(anova_result)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## Model          2  1.0267   0.5133    3054 <2e-16 ***
## Residuals    297  0.0499   0.0002
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

What does it reveal?

The ANOVA results reveal a significant difference in test errors across the three models, as indicated by a high F-statistic of 3054. The very small p-value ($< 2e-16$) rejects the null hypothesis, confirming that the variability in test errors between the models is much greater than within the models. This suggests that the models are not equally effective. The optimal model is likely the best-performing one, as its test error differs significantly from the simpler and overly complex models.

2. Obtain and plot the 95% confidence and prediction bands for the dataset D_n

```
optimal_degree <- 10

poly_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE))

# Generate predictions and intervals for the entire dataset
x_range <- seq(min(x), max(x), length.out = 100)
predictions <- predict(poly_model, newdata = data.frame(x = x_range),
                      interval = "confidence", level = 0.95)
prediction_intervals <- predict(poly_model, newdata = data.frame(x = x_range),
                              interval = "prediction", level = 0.95)

# Plot the data points
plot(x, y, main = paste("Polynomial Regression with 95% Confidence and Prediction Bands"),
     xlab = "x", ylab = "y", pch = 16, col = "blue")

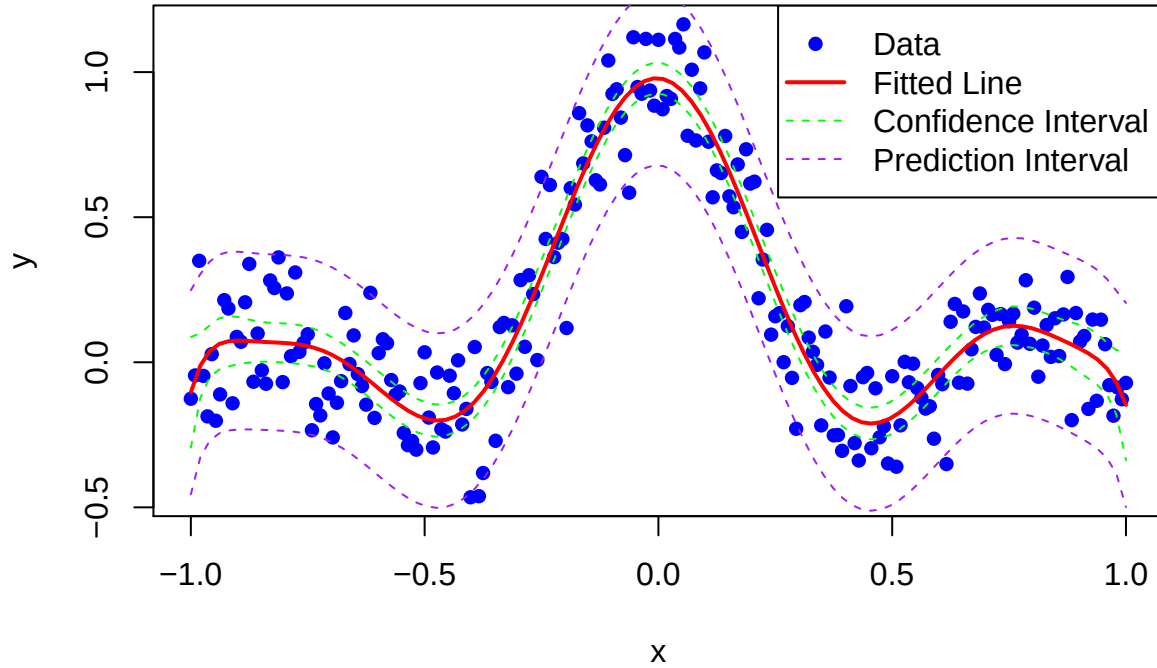
# Plot the fitted polynomial regression line
lines(x_range, predictions[, "fit"], col = "red", lwd = 2)

# Plot the 95% confidence interval
lines(x_range, predictions[, "lwr"], col = "green", lty = 2)
lines(x_range, predictions[, "upr"], col = "green", lty = 2)

# Plot the 95% prediction interval
lines(x_range, prediction_intervals[, "lwr"], col = "purple", lty = 2)
lines(x_range, prediction_intervals[, "upr"], col = "purple", lty = 2)

legend("topright", legend = c("Data", "Fitted Line", "Confidence Interval", "Prediction Interval"),
     col = c("blue", "red", "green", "purple"), pch = c(16, NA, NA, NA),
     lty = c(NA, 1, 2, 2), lwd = c(NA, 2, 1, 1))
```


Polynomial Regression with 95% Confidence and Prediction Bands



3. Write the mathematical expression for:

The confidence band for a single observation (X_i , Y_i)

The confidence band gives the interval around the predicted value, which reflects the uncertainty of the estimated regression line. The confidence interval for the estimated mean response $\hat{Y}(X_i)$ at a given point X_i is:

$$\hat{Y}(X_i) \pm t_{\alpha/2, n-p} \cdot \sqrt{\hat{\sigma}^2 (X_i^T (X^T X)^{-1} X_i)}$$

Where:

- $\hat{Y}(X_i)$ is the predicted value at X_i ,
- $t_{\alpha/2, n-p}$ is the critical value from the t -distribution with $n - p$ degrees of freedom, where n is the sample size and p is the number of parameters in the model,
- $\hat{\sigma}^2$ is the estimated variance of the errors,
- X_i is the vector of the i -th observation's predictor variable(s),
- X is the design matrix of all predictor values.

The prediction band for a single observation (X_i , Y_i)

The prediction band, on the other hand, accounts for both the uncertainty in the regression line and the inherent variability in the response variable Y_i . The prediction interval for a new observation at X_i is:

$$\hat{Y}(X_i) \pm t_{\alpha/2, n-p} \cdot \sqrt{\hat{\sigma}^2 (1 + X_i^T (X^T X)^{-1} X_i)}$$

Where:

- $\hat{Y}(X_i)$ is the predicted value at X_i ,
- $t_{\alpha/2, n-p}$ is the critical value from the t -distribution,
- $\hat{\sigma}^2$ is the estimated variance of the errors,
- X_i is the vector of the i -th observation's predictor variable(s),
- X is the design matrix of all predictor values,
- The term $1 + X_i^T(X^T X)^{-1}X_i$ accounts for both the variance of the estimate of $\hat{Y}(X_i)$ and the variability of the new observation Y_i .

4. Comment extensively on what the confidence and prediction bands reveal about the model

The confidence band and prediction band offer different types of uncertainty estimates for a regression model's predictions:

- **Confidence Band:**
 - Represents the uncertainty about the *estimated regression line*.
 - Tells us where the *true regression line* would likely fall, based on the data, at each value of X_i .
 - A *narrow confidence band* indicates that the model's predictions are reliable. A *wider band* suggests more uncertainty in the regression estimate.
 - Influenced by sample size, model complexity, and variability in the predictor X .
- **Prediction Band:**
 - Represents the uncertainty about where a *new observation* Y_i will fall for a given X_i .
 - The prediction band accounts for both *model error* and *inherent variability* in the data.
 - A *wider prediction band* reflects more uncertainty in predicting new values.
 - A *narrow prediction band* suggests high precision in predictions, while a *wide band* indicates high uncertainty.
- **Key Differences:**
 - The confidence band focuses on the uncertainty of the regression model itself.
 - The prediction band focuses on the uncertainty in predicting new data points, which is why it is broader.
- **Model Evaluation:**
 - A *narrow confidence band* suggests a reliable model.
 - A *narrow prediction band* suggests the model will make precise predictions for new data.
 - An *overly complex model* may result in a narrow confidence band but a wide prediction band, indicating overfitting and poor generalization.

In conclusion, the confidence bands help assess the accuracy of the estimated relationship between X and Y , while prediction bands give insight into the model's ability to predict future data.

Exercise 2:

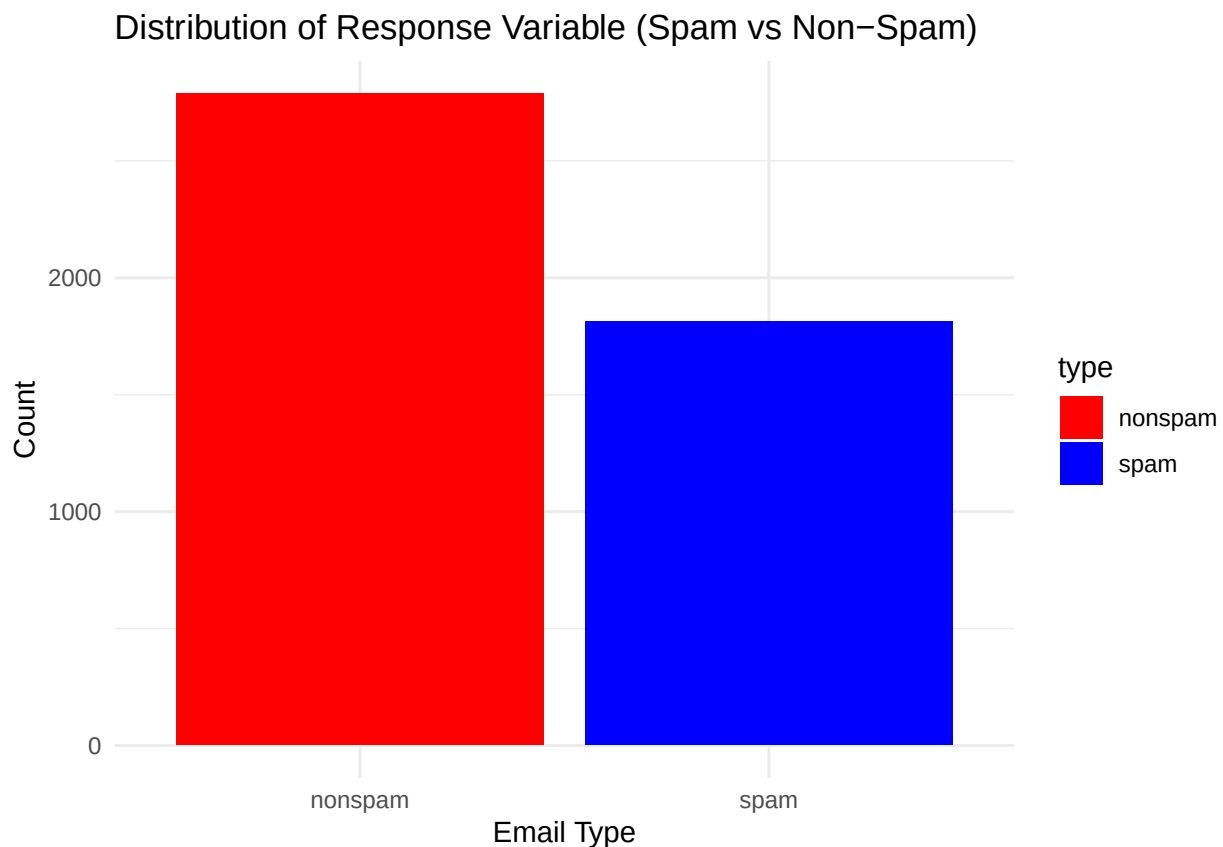
```
data(spam)
```

1. Plot the distribution of the response for this dataset and comment.

```
table(spam$type)
```

```
##  
## nonspam    spam  
##    2788    1813
```

```
ggplot(spam, aes(x = type, fill = type)) +  
  geom_bar() +  
  labs(title = "Distribution of Response Variable (Spam vs Non-Spam)",  
        x = "Email Type",  
        y = "Count") +  
  scale_fill_manual(values = c("red", "blue")) +  
  theme_minimal()
```



comment:

- **Class Imbalance:** The dataset is imbalanced, with significantly more non-spam emails than spam emails. This imbalance can impact the performance of classification models.
- **Data Composition:** Approximately 75% of the emails are non-spam, while 25% are spam.

2. Comment on the shape of this dataset in terms of the sample size and the dimensionality of the input space

```
sample_size <- nrow(spam)
dimensions <- ncol(spam) - 1

cat("Sample Size:", sample_size, "\n")
```

```
## Sample Size: 4601
```

```
cat("Dimensionality of Input Space:", dimensions, "\n")
```

```
## Dimensionality of Input Space: 57
```

The sample size is reasonable, but the dimensionality (with 57 features) presents challenges typically associated with high-dimensional datasets, such as the risk of overfitting and difficulties in learning discriminative boundaries. However, these challenges can often be mitigated by using dimensionality reduction techniques, careful model selection, or regularization.

3. Comment succinctly from the statistical perspective on the type of data in the input space

```
str(spam)
```

```
## 'data.frame':    4601 obs. of  58 variables:
## $ make           : num  0 0.21 0.06 0 0 0 0 0 0.15 0.06 ...
## $ address        : num  0.64 0.28 0 0 0 0 0 0 0 0.12 ...
## $ all            : num  0.64 0.5 0.71 0 0 0 0 0 0.46 0.77 ...
## $ num3d          : num  0 0 0 0 0 0 0 0 0 0 ...
## $ our            : num  0.32 0.14 1.23 0.63 0.63 1.85 1.92 1.88 0.61 0.19 ...
## $ over           : num  0 0.28 0.19 0 0 0 0 0 0 0.32 ...
## $ remove         : num  0 0.21 0.19 0.31 0.31 0 0 0 0.3 0.38 ...
## $ internet       : num  0 0.07 0.12 0.63 0.63 1.85 0 1.88 0 0 ...
## $ order          : num  0 0 0.64 0.31 0.31 0 0 0 0.92 0.06 ...
## $ mail           : num  0 0.94 0.25 0.63 0.63 0 0.64 0 0.76 0 ...
## $ receive        : num  0 0.21 0.38 0.31 0.31 0 0.96 0 0.76 0 ...
## $ will           : num  0.64 0.79 0.45 0.31 0.31 0 1.28 0 0.92 0.64 ...
## $ people         : num  0 0.65 0.12 0.31 0.31 0 0 0 0 0.25 ...
## $ report         : num  0 0.21 0 0 0 0 0 0 0 0 ...
## $ addresses      : num  0 0.14 1.75 0 0 0 0 0 0 0.12 ...
## $ free           : num  0.32 0.14 0.06 0.31 0.31 0 0.96 0 0 0 ...
## $ business       : num  0 0.07 0.06 0 0 0 0 0 0 0 ...
## $ email          : num  1.29 0.28 1.03 0 0 0 0.32 0 0.15 0.12 ...
## $ you            : num  1.93 3.47 1.36 3.18 3.18 0 3.85 0 1.23 1.67 ...
## $ credit         : num  0 0 0.32 0 0 0 0 0 3.53 0.06 ...
## $ your           : num  0.96 1.59 0.51 0.31 0.31 0 0.64 0 2 0.71 ...
## $ font           : num  0 0 0 0 0 0 0 0 0 0 ...
## $ num000         : num  0 0.43 1.16 0 0 0 0 0 0 0.19 ...
```

```

## $ money      : num  0 0.43 0.06 0 0 0 0 0 0.15 0 ...
## $ hp         : num  0 0 0 0 0 0 0 0 0 0 ...
## $ hpl        : num  0 0 0 0 0 0 0 0 0 0 ...
## $ george     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ num650     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ lab        : num  0 0 0 0 0 0 0 0 0 0 ...
## $ labs       : num  0 0 0 0 0 0 0 0 0 0 ...
## $ telnet     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ num857     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ data       : num  0 0 0 0 0 0 0 0 0.15 0 ...
## $ num415     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ num85      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ technology : num  0 0 0 0 0 0 0 0 0 0 ...
## $ num1999    : num  0 0.07 0 0 0 0 0 0 0 0 ...
## $ parts      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ pm         : num  0 0 0 0 0 0 0 0 0 0 ...
## $ direct     : num  0 0 0.06 0 0 0 0 0 0 0 ...
## $ cs         : num  0 0 0 0 0 0 0 0 0 0 ...
## $ meeting    : num  0 0 0 0 0 0 0 0 0 0 ...
## $ original   : num  0 0 0.12 0 0 0 0 0 0.3 0 ...
## $ project    : num  0 0 0 0 0 0 0 0 0 0.06 ...
## $ re         : num  0 0 0.06 0 0 0 0 0 0 0 ...
## $ edu        : num  0 0 0.06 0 0 0 0 0 0 0 ...
## $ table      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ conference : num  0 0 0 0 0 0 0 0 0 0 ...
## $ charSemicolon : num  0 0 0.01 0 0 0 0 0 0 0.04 ...
## $ charRoundbracket : num  0 0.132 0.143 0.137 0.135 0.223 0.054 0.206 0.271 0.03 ...
## $ charSquarebracket : num  0 0 0 0 0 0 0 0 0 0 ...
## $ charExclamation : num  0.778 0.372 0.276 0.137 0.135 0 0.164 0 0.181 0.244 ...
## $ charDollar  : num  0 0.18 0.184 0 0 0 0.054 0 0.203 0.081 ...
## $ charHash    : num  0 0.048 0.01 0 0 0 0 0 0.022 0 ...
## $ capitalAve  : num  3.76 5.11 9.82 3.54 3.54 ...
## $ capitalLong : num  61 101 485 40 40 15 4 11 445 43 ...
## $ capitalTotal : num  278 1028 2259 191 191 ...
## $ type        : Factor w/ 2 levels "nonspam","spam": 2 2 2 2 2 2 2 2 2 2 ...

```

From a statistical perspective, the dataset’s input space (57 features) can be characterized as follows:

- **Continuous Variables:** Most features (e.g., make, address, all, etc.) appear to be continuous numerical values, representing frequencies or proportions of certain words or characters in the emails. **Range and Distribution:** These variables likely have varying ranges and distributions, which may require normalization or scaling for effective modeling. **Categorical Feature:**
- **Target Variable (type):** The target variable is categorical with two levels ("nonspam" and "spam"), indicating a binary classification problem. **Sparse Features:**
- **Zero-Inflated:** Many features have a high proportion of zeros (e.g., num3d, report, addresses), indicating sparsity. This sparsity can affect model performance and may require techniques like regularization or feature selection. **Character and Capitalization Metrics:**
- **Character-Specific Features:** Features like charSemicolon, charRoundbracket, etc., capture the frequency of specific characters, providing insights into the email’s structure and content. **Capitalization Metrics:** Features like capitalAve, capitalLong, and capitalTotal measure the average, longest streak, and total number of capital letters, which can be indicative of spam emails.

- 57 Dimensions: The high dimensionality relative to the sample size (4601) poses challenges such as the curse of dimensionality and increased risk of overfitting. Dimensionality reduction techniques may be beneficial. write in latex form

4. Using the whole data for training and the whole data for test, building the above four learning machines, then plot the comparative ROC curves on the same grid

```
# Load the spam dataset
data(spam, package = "kernlab")
X <- spam[, -58] # Features (excluding the target column)
y <- spam[, 58]  # Target variable

# Convert target variable to binary
y <- as.numeric(y) - 1

# Standardize the features
scaler <- preProcess(X, method = c("center", "scale"))
X <- predict(scaler, X)

# Fit models
lda_model <- lda(X, grouping = y)      # LDA
qda_model <- qda(X, grouping = y)      # QDA
nb_model <- naiveBayes(X, y)           # Naive Bayes

# Fisher's Linear Discriminant (FLD)
fld_projection <- as.matrix(X) %*% lda_model$scaling      # Project data onto FLD axis
fld_min <- min(fld_projection)
fld_max <- max(fld_projection)
y_pred_fld_prob <- (fld_projection - fld_min) / (fld_max - fld_min) # Normalize to probabilities

# Predict probabilities for LDA, QDA, and Naive Bayes
y_pred_lda <- predict(lda_model, X)$posterior[, 2]
y_pred_qda <- predict(qda_model, X)$posterior[, 2]
y_pred_nb <- predict(nb_model, X, type = "raw")[, 2]

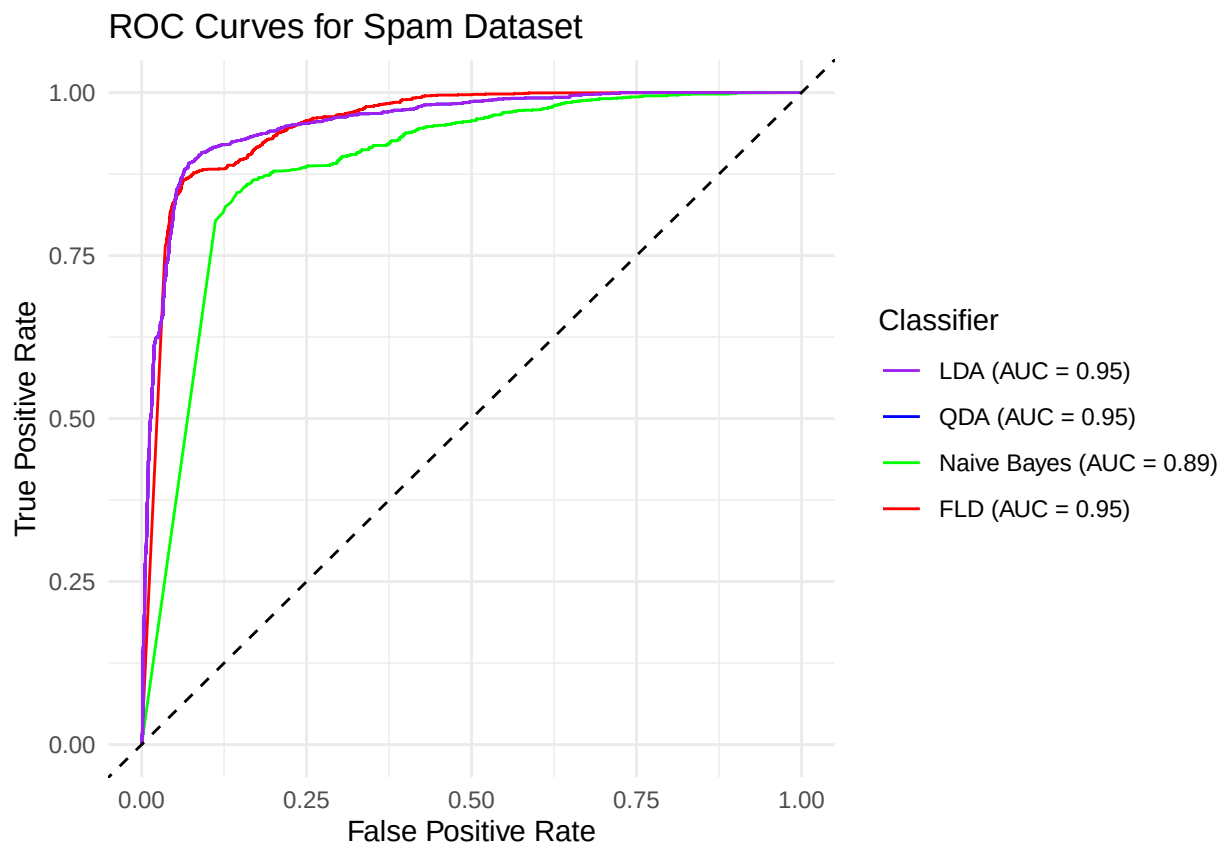
# Calculate ROC and AUC
roc_lda <- roc(y, y_pred_lda, quiet = TRUE)
roc_qda <- roc(y, y_pred_qda, quiet = TRUE)
roc_nb <- roc(y, y_pred_nb, quiet = TRUE)
roc_fld <- roc(y, y_pred_fld_prob, quiet = TRUE)

# Plot ROC curves
ggplot() +
  geom_line(aes(x = 1 - roc_lda$specificities, y = roc_lda$sensitivities, color = "LDA")) +
  geom_line(aes(x = 1 - roc_qda$specificities, y = roc_qda$sensitivities, color = "QDA")) +
  geom_line(aes(x = 1 - roc_nb$specificities, y = roc_nb$sensitivities, color = "Naive Bayes")) +
  geom_line(aes(x = 1 - roc_fld$specificities, y = roc_fld$sensitivities, color = "FLD")) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  labs(
    title = "ROC Curves for Spam Dataset",
    x = "False Positive Rate",
```

```

y = "True Positive Rate"
) +
scale_color_manual(
  name = "Classifier",
  values = c("LDA" = "blue", "QDA" = "red", "Naive Bayes" = "green", "FLD" = "purple"),
  labels = c(
    paste("LDA (AUC = ", round(auc(roc_lda), 2), ")"), sep = """,
    paste("QDA (AUC = ", round(auc(roc_qda), 2), ")"), sep = """,
    paste("Naive Bayes (AUC = ", round(auc(roc_nb), 2), ")"), sep = """,
    paste("FLD (AUC = ", round(auc(roc_fld), 2), ")"), sep = """)
  )
) +
theme_minimal()

```



5. Comment succinctly on what the ROC curves reveal for this data and argue in light of the theory whether or not that was to be expected.

The ROC curves for the spam dataset illustrate the trade-off between true positive rate (sensitivity) and false positive rate (specificity) for different classification models. Key Observations:

- **LDA, QDA, and FLD** have near-perfect performance, with ROC curves closely hugging the top-left corner. This indicates high sensitivity and specificity, suggesting excellent discrimination between spam and non-spam emails.
- **Naive Bayes** performs slightly worse, with a lower AUC. This could be due to its assumption of feature independence, which may not hold true for real-world text data.

Theoretical Expectations: - **LDA**, **QDA**, and **FLD** are generally robust classifiers, especially when the assumptions underlying their models are met. In the case of spam classification, where features might be correlated, LDA and FLD can still perform well due to their ability to handle overlapping distributions. - **Naive Bayes** can be surprisingly effective even when its assumptions are violated. However, in text classification, where features (words) are often correlated, its performance might degrade compared to more sophisticated models.

Conclusion: Overall, the ROC curves confirm that LDA, QDA, and FLD are highly effective for spam classification in this dataset, while Naive Bayes, despite its simplicity, still provides reasonable performance.

6. Using `set.seed(19671210)` along with a 2/3 training 1/3 test in the context stratified stochastic holdout split of the data, compute $S = 50$ replications of the test error for all the above learning machines.

```
set.seed(19671210)

# Initialize storage for test errors
errors <- data.frame(
  Replication = 1:50,
  LDA = numeric(50),
  QDA = numeric(50),
  NaiveBayes = numeric(50),
  FLD = numeric(50) # Add column for FLD errors
)

# Ensure y_train is a factor with correct levels (0 and 1 for binary classification)
y <- factor(y, levels = c(0, 1)) # Ensure y is a factor with 0 and 1 levels
zero_var_cols <- apply(X, 2, function(x) var(x) == 0)
X_cleaned <- X[, !zero_var_cols]
# Run 50 replications
for (i in 1:50) {
  # Stratified split into training (2/3) and testing (1/3)
  trainIndex <- createDataPartition(y, p = 2/3, list = FALSE)
  X_train <- X_cleaned[trainIndex, ]
  y_train <- y[trainIndex]
  X_test <- X_cleaned[-trainIndex, ]
  y_test <- y[-trainIndex]

  # Apply PCA to reduce dimensionality (using the first 10 principal components)
  pca <- prcomp(X_train, center = TRUE, scale. = TRUE)
  X_train_pca <- as.data.frame(pca$x[, 1:10]) # First 10 principal components for training
  X_test_pca <- as.data.frame(predict(pca, X_test)[, 1:10]) # First 10 principal components for testing

  # Train the models using PCA-transformed data
  lda_model <- lda(y_train ~ ., data = data.frame(X_train_pca, y_train))
  qda_model <- qda(y_train ~ ., data = data.frame(X_train_pca, y_train))
  nb_model <- naiveBayes(y_train ~ ., data = data.frame(X_train_pca, y_train))

  # Calculate FLD projection probabilities
  fld_projection <- as.matrix(X_test_pca) %*% lda_model$scaling
  fld_min <- min(fld_projection)
  fld_max <- max(fld_projection)
```



```

fld_probabilities <- (fld_projection - fld_min) / (fld_max - fld_min) # Normalize probabilities

# FLD predictions
fld_pred <- ifelse(fld_probabilities > 0.5, levels(y_train)[2], levels(y_train)[1]) # Binary predictions

# Make predictions
lda_pred <- predict(lda_model, X_test_pca)$class
qda_pred <- predict(qda_model, X_test_pca)$class
nb_pred <- predict(nb_model, X_test_pca) # Naive Bayes predictions

# Store test errors in the dataframe
errors$LDA[i] <- mean(lda_pred != y_test)
errors$QDA[i] <- mean(qda_pred != y_test)
errors$NaiveBayes[i] <- mean(nb_pred != y_test)
errors$FLD[i] <- mean(fld_pred != y_test)
}

# View the first 15 rows of the error dataframe
head(errors, 15)

```

##	Replication	LDA	QDA	NaiveBayes	FLD
## 1	1	0.1213307	0.2198304	0.1813438	0.2615786
## 2	2	0.1285062	0.1480757	0.1454664	0.3339856
## 3	3	0.1285062	0.2061318	0.1780822	0.1272016
## 4	4	0.1239400	0.2354860	0.1643836	0.2537508
## 5	5	0.1376386	0.2237443	0.1454664	0.3535551
## 6	6	0.1304631	0.1611220	0.1298108	0.3770385
## 7	7	0.1461187	0.1611220	0.1382909	0.3333333
## 8	8	0.1356817	0.2367906	0.1617743	0.2818004
## 9	9	0.1343770	0.2002609	0.1382909	0.3757339
## 10	10	0.1167645	0.2341813	0.1526419	0.3131115
## 11	11	0.1337247	0.2361383	0.1702544	0.2178735
## 12	12	0.1245923	0.2407045	0.1630789	0.1800391
## 13	13	0.1272016	0.2263536	0.1650359	0.2315721
## 14	14	0.1298108	0.2198304	0.1761252	0.2583170
## 15	15	0.1461187	0.1800391	0.1559035	0.3535551

7. Plot the comparative boxplots (be sure to properly label the plots)

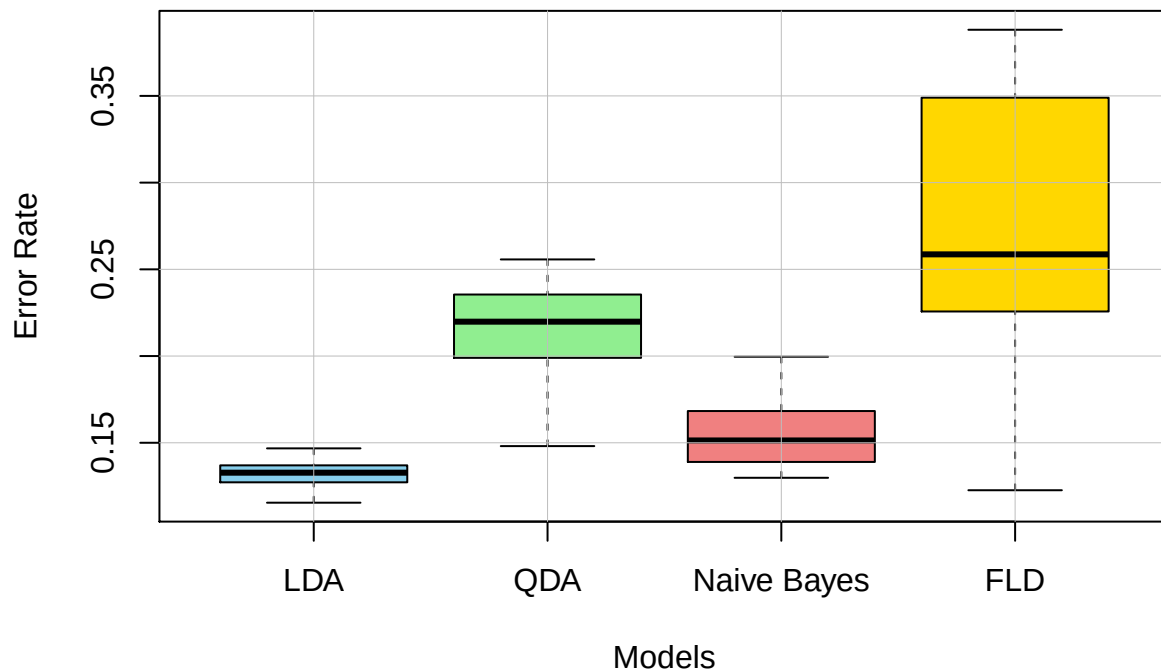
```

# Create comparative boxplots
boxplot(
  errors$LDA, errors$QDA, errors$NaiveBayes, errors$FLD,
  names = c("LDA", "QDA", "Naive Bayes", "FLD"),
  main = "Comparative Boxplots of Test Errors",
  ylab = "Error Rate",
  xlab = "Models",
  col = c("skyblue", "lightgreen", "lightcoral", "gold"),
  border = "black"
)

# Add a grid for better visualization
grid(nx = NULL, ny = NULL, lty = 1, col = "gray", lwd = 0.5)

```

Comparative Boxplots of Test Errors



The boxplots reveal the relationship between test error rates and model complexity:

- LDA (Linear Discriminant Analysis): Exhibits the lowest test error with minimal variability, attributed to its simplicity, assuming linear decision boundaries and shared covariance matrices.
- QDA (Quadratic Discriminant Analysis): Shows higher test error and greater variability due to increased complexity from modeling quadratic decision boundaries with separate covariance matrices. This makes it more prone to overfitting.
- Naive Bayes : Achieves moderate error with low variability, benefiting from its simplicity (independence assumption) but sacrificing accuracy by failing to capture feature interactions.
- FLD (Flexible Discriminant Analysis): Has the highest test error and variability, reflecting overfitting from its high flexibility and complexity.

In summary, simpler models such as LDA and Naive Bayes generalize better, while more complex models like QDA and FLD are more error-prone due to their sensitivity to noise and limited data

Exercise 3:

Watch this YouTube video